Summary Generic Schema Matching with Cupid

By Ossama Sijbesma (9890238) – 17 October 2026

Schema matching is an operation that takes two schemas, such as the ones in figure 1, and returns a mapping from one element to another based on a similarity match. In figure 1 a mapping might pair *Lines.Item.Qty* with *Items.Item.Quantity*. Schema matching is used in many applications where there is a need for such mappings. From a professional point of view, I could see this being used for related data discovery within large data platforms. Within the paper they also mention mapping messages across different file formats (XML, Json, etc.), consolidating different data sources into a unified data warehouse, and identifying integration points between heterogeneous databases for mediators.

The main problem

At the time that the paper was written, schema matching was primarily performed as a manual task by domain experts. There was some automated tooling, but all it could do was match schema elements with the same name and same structure.

PO	POrder
Lines	Items
Item	Item
Line	ItemNumber
Qty	Quantity
Uom	UnitOfMeasure

Figure 1, Two schemas to be matched

The authors also mention that schema matching is tricky. Even when two schemas deal with the same concept, they often have different structures and different names (see figure 1). There is also the issue that the same word can have different meanings depending on the context.

Furthermore, schema matching is subjective. Schemas don't always clearly describe the data they store (semantics). And two schemas can often be connected in multiple ways. Therefore, it is preferable to have the real-world context documented in some way so it can be referred to, e.g. a thesaurus containing enforced company standards, domain language, etc.

The proposed solution

The authors propose a schema matching component called Cupid, which should be able to perform general-purpose schema matching on different data sources by only considering schema information and not instance data (schema based).

Cupid tackles the problem by calculating multiple similarity scores between schema elements and deriving a mapping from those scores. It does this by representing schemas as trees with the schema elements being the leaf nodes. Cupid performs the following phases:

Phase 1 Linguistic matching; pairs of schema elements are matched based on their names, data types, and domains resulting in a linguistic similarity coefficient ($lsim$). This coefficient has a value in the range of [0,1] of which 1 is considered a perfect score. It has the following steps:

1. Normalization: We perform tokenization, expansion, and elimination. A thesaurus is used to look up general vocabulary and domain specific terms.
2. Categorization: Schema elements are clustered into categories based on their data types, schema hierarchy, and linguistic content. Elements can belong to multiple categories.
3. Comparison: $lsim$ is computed by performing sub-string matching on the tokens originating from the name synonym, or hypernymy (thesaurus) of the schema elements.

Phase 2 Structural matching; for every pair of schema elements, we calculate the similarity of their respective contexts. This results in the structural similarity ($ssim$), which also is a value between [0,1]. Ancestor similarity directly impacts this score; highly similar ancestors elevate it while low similarity ancestors lower it. This is configurable using parameters.

Phase 3 Weighted similarity & mapping; using $lsim$ and $ssim$ the weighted similarity is calculated as follows: $wsim = w_{struct} \times ssim + (1 - w_{struct}) \times lsim$, where w_{struct} is in the range [0,1]. Final mappings are created by selecting pairs of schema elements with the highest (based on threshold) weighted similarity values. This can result in a 1:1 or $1{:}\,n$ cardinality.

Any optimization techniques or special cases discussed

Among the paper's optimization techniques, lazy expansion and pruning stands out for me as computational improvements. Pruning limits the tree size and depth, whereas lazy expansion prevents duplicate comparisons by comparing elements in the graph before tree conversion.

As a special case, Cupid interprets reverential constraints as potential join view. For each foreign key, we introduce a node that represents the join of the participating tables. This will have a positive effect on structural similarity but can cause the tree to grow a lot extra.

The strengths and limitations of Cupid

A major strength of Cupid is that it works on many different data models. It does this by changing (non-hierarchical) schemas into a graph first and then into a tree on which the phases are executed. Another nice part about using trees is that it allows for context dependent mapping during the structural matching (basically including the parent node).

Schema tree construction fails if a schema contains a cyclic relationship as it turns into an infinite recursion. Cupid is also dependent on a thesaurus for linguistic matching, if dropped the results seem to be worse or unchanged.

The evaluation methodology

The authors evaluated Cupid against two other schema-based prototypes, DIKE and MOMIS, by comparing their performance on canonical examples that isolate specific matching properties and real-world examples. A quantitative comparison of Cupid, DIKE, and MOMIS is not possible because schema matching is subjective, and Dike and MOMIS were designed for another use case (schema integration). The performance was therefore assessed qualitatively.